TP4 : Le Colisée et la Boîte à Meuh (5 heures - 2 séances)

— Tous les TPs sont à réaliser **individuellement**.
— **L'utilisation d'une IA générative en séance de TP est strictement interdite**, si vous avez des questions, posez-les à l'enseignant(e) présent(e) dans la salle.
— **Tous les TPs doivent être effectués sous Linux, en utilisant Emacs**.
— Le temps prévu en cours pour chaque TP est volontairement limité : il ne permet pas, en général, de terminer l'ensemble du sujet.
  - Il est **indispensable** de poursuivre le travail chez vous pour finaliser les exercices.
  - Terminer les TPs vous aidera à progresser efficacement, à consolider vos acquis et à être prêt pour les évaluations.
  - En cas de difficulté, n'hésitez pas à poser vos questions à la séance suivante.

1 - Observez un processus ($\simeq$10 min)

1. Retrouvez, à l'aide de la commande `ps`, le PID du processus bash.
2. Ajoutez l'option `-l` à la commande précédente, puis en utilisant l'option `-p` trouvez le nom du processus parent du processus bash.
3. Utilisez maintenant la commande `pstree` afin de trouver le PID du processus bash ainsi que le nom de son processus père.
4. Le répertoire `/proc/<PID>` contient des fichiers décrivant l'état du processus de PID `<PID>`. Trouvez le fichier contenant la ligne de commande ayant servi à lancer le processus `bash`.
5. Consultez la documentation de la commande `yes`. À quoi sert-elle ?
   Ouvrez un terminal et lancez la commande suivante : `yes "Unix, c'est super"`
   Pendant l'exécution de la commande `yes`, lancez la commande top dans un autre terminal. Quels sont les processus qui consomment le plus de CPU ?

2 - Suspendre un processus($\simeq$15 min)

Lancement en avant-plan

1. Tapez `emacs` pour lancer, en avant-plan, le processus emacs.
   Pour quelle raison l'invite de commande (prompt) n'est plus présente dans le terminal ? Essayez de taper la commande `ls` dans le terminal et observez le résultat.
2. Dans le terminal, appuyez `control+z` pour suspendre le processus en avant-plan (sans le détruire) et ainsi pouvoir utiliser le terminal. Dans le terminal, vérifiez que vous pouvez exécuter la commande `ls`. Dans la fenêtre d'Emacs, tapez quelques caractères pour vérifier son fonctionnement.
3. Lancez la commande `ps -l` pour identifier le processus qui s'exécute et connaître l'état des autres processus lancés depuis ce terminal. Consultez la documentation de la commande `ps` pour trouver cette information (section "Codes d'état de processus", ou "Process state codes").
4. Tapez la commande `fg` pour relancer le processus `emacs` en avant-plan. Pouvez-vous maintenant écrire dans la fenêtre d'Emacs ? Dans le terminal ?

Lancement en arrière-plan

1. Si ce n'est pas déjà fait, tapez la commande `emacs` pour lancer, en avant-plan, le processus emacs.
2. Dans le terminal, appuyez sur `control+z` afin de suspendre le processus emacs.
3. Vérifiez avec ps que le processus emacs est bien suspendu.
4. Tapez la commande `bg` pour relancer le processus emacs en arrière-plan. Pouvez-vous maintenant écrire dans la fenêtre d'Emacs ? Dans le terminal ? Quelle est la différence entre les commandes fg et bg ?

5. Utilisez la commande ps pour afficher l'état du processus emacs.
6. Fermez le programme emacs, puis relancez-le en terminant la ligne de commande par & : `emacs &`. Vérifiez que le terminal reste utilisable. Vérifiez avec `ps` que l'état du processus emacs est le même qu'à la question précédente. Lors du lancement du processus emacs à la question précédente, deux nombres sont apparus sur le terminal. À quoi correspondent-ils ?

3 - Tuer des processus (≃10min)

Tuer un processus

1. Lancez 2 processus `xeyes` en arrière-plan.
2. Retrouvez le PID du premier processus `xeyes` lancé et utilisez la commande `kill` pour le détruire (aidez-vous de la documentation et du cours)
3. Vérifiez que le processus n'existe plus.

Tuer plusieurs processus

1. Relancez un processus `xeyes` en arrière-plan et vérifiez qu'il existe bien 2 processus `xeyes` (puisque vous n'en avez détruit qu'un seul.
2. Utilisez la commande `killall` pour terminer tous les processus `xeyes` en cours d'exécution (aidez-vous de la documentation).

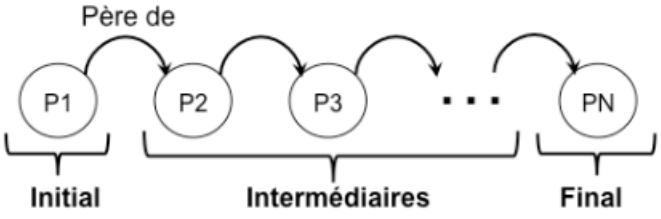
Objectifs de la suite

Le but de la suite de ce TP est de vous faire manipuler les signaux, les tubes et les redirections. Il se décompose en 2 parties (rédiger chacune des 2 parties dans un dossier distinct) :

1. Chaîne de processus :
   Création de plusieurs processus en utilisant un algorithme récursif.
2. Ave César :
   Manipulation des signaux en concevant un (tout petit) Colisée de Rome à partir de la chaîne de processus : Le Colisée est constitué de gladiateurs qui saluent César, et de César qui les trucide dans la joie.
3. La boîte à Meuh :
   Transformer le Colisée en boîte à Meuh, c'est-à-dire que les processus vont afficher Meuh les uns après les autres.

4 - Chaîne de processus (≃1h10)

Le but est d'apprendre à créer plusieurs processus en utilisant un algorithme récursif. Comme présenté sur la figure ci-dessous, vous allez créer une chaîne de N processus. Dans une chaîne de processus, chaque processus est le père du suivant, sauf pour le dernier processus de la chaîne qui, lui, n'a pas de fils. Le premier processus est appelé le processus initial et le dernier le processus final. Les autres processus sont les processus intermédiaires.



L'algorithme à écrire est récursif. Si N vaut 1, alors l'unique processus de la chaîne est le processus final. Il ne doit donc rien faire de spécial. Si N est strictement supérieur à 1, alors le processus doit encore créer N-1 processus. Pour cela, il lui suffit de lancer le script que vous allez créer (`chaine.sh`) avec comme paramètre N-1.

1. Écrire le script qui crée une chaîne de processus étape par étape. Pour commencer, créer un script nommé `chaine.sh` qui :
   — Vérifie ses arguments. Si le script ne reçoit pas un unique argument, il doit quitter avec un message d'erreur et un code de retour faux (-1).
   — Affiche le message "`Processus X démarre`, où `X` est le PID du processus.
   — Quitte avec un code de retour vrai (0).
2. Avant de créer récursivement les processus, dans cette question, vous devez affichez ce que l'algorithme est supposé faire. Modifier votre script de façon à ce que :
   — si N est strictement supérieur à 1, votre script affiche "`Il reste K processus à créer`, où `K` est égal à N-1,
   — sinon, votre script affiche "`Fin de chaîne`".
3. Finaliser l'algorithme : Après avoir affiché "`Il reste K processus à créer`", où `K`=N-1, votre script doit lancer la commande `./chaine.sh K` pour créer le processus suivant de la chaîne. Vérifiez que vous démarrez bien 4 processus lorsque vous invoquez `./chaine.sh 4` (en dehors de votre script, vous pouvez utiliser la commande `ps ux` pour lister les processus de l'utilisateur, n'hésitez pas à utiliser un `grep` pour filtrer l'affichage pour ne visualiser que les processus lancés avec la commande `./chaine.sh`).
4. Il faut maintenant lancer les processus en tâche de fond. Un processus ne doit aussi se terminer que lorsque son fils direct est terminé. Enfin, tous les processus (y compris le processus final) doivent afficher "`Processus X termine`", où `X` est le PID du processus courant. Pour les processus qui possèdent un fils, cet affichage doit avoir lieu après avoir attendu la fin du fils.
   Indication : utiliser la commande `wait` avec le PID du fils, ajoutez un `sleep` de 1 seconde après le `wait` afin de visualiser ce qu'il se passe.
   Modifier votre script en conséquence. Vérifiez aussi que les messages de terminaison s'affichent dans l'ordre inverse des créations en lançant `./chaine.sh 4`
5. Nous souhaitons maintenant communiquer le PID du processus initial jusqu'au processus final. Le langage Bash fournit un mécanisme pour connaître le PID de son parent direct, mais pas de ses autres aïeuls. Il faut donc communiquer ce PID via un nouveau mécanisme. Pour mettre en œuvre un tel mécanisme, nous devons résoudre deux sous-problèmes :
   — Il faut être capable de communiquer le PID du processus initial jusqu'au processus final. Pour cela, nous utilisons une variable nommée `pidInitial`, initialisée par le processus initial à son PID, et exportée jusqu'au processus final.
   — Il faut que le processus initial soit capable de se reconnaître de façon à initialiser et exporter cette variable `pidInitial`. Tous les processus, sauf le processus initial, vont démarrer avec une copie de la variable `pidInitial` puisque cette dernière est exportée par le processus initial. Le processus initial est donc l'unique processus pour lequel la variable `pidInitial` n'est pas définie initialement, c.-à-d., pour lequel la variable `pidInitial` a pour valeur la chaîne de caractères vide, qui se reconnaît avec la construction `[[ -z "$pidInitial" ]]`).
   Modifier votre script de façon à ce que le processus initial initialise et exporte la variable `pidInitial`.
   Indication : pensez à utiliser la commande `export`.
   Pour vous assurer que votre programme est correct, modifier aussi l'affichage "`Processus X démarre`", où `X` est le PID du processus courant, en " `Processus X démarre avec le processus initial Y`", où `Y` est le PID du processus initial. Testez votre programme en lançant `./chaine.sh 4`

Vous venez de créer votre première application multi-processus complexe !

5 - Ave César (≃1h30)

Ici, le but est de manipuler les signaux. Pour cela, vous allez créer un (tout petit) Colisée de Rome à partir de la chaîne de processus le partie suivante. Le Colisée est constitué de gladiateurs qui saluent César, et de César qui les trucide dans la joie.

5.1 - Création des gladiateurs

Un gladiateur est un processus qui affiche toutes les 5 secondes `X: Ave César`, où `X` est le PID du gladiateur, et qui meurt en criant "`X: Morituri te salutant`" lorsqu'il reçoit un signal `USR1`.

1. Copiez le script de l'exercice sur les chaînes de processus en `gladiateur.sh`. Dans votre script, au lieu d'attendre la mort d'un enfant, d'afficher "`Processus X termine`" et de retourner un code d'erreur vrai, `gladiateur.sh` doit maintenant exécuter une boucle infinie qui affiche toutes les 5 secondes "`X: Ave César`", où `X` est le PID du gladiateur.
   Remarque Dans cette sous-partie, vous devez systématiquement tester votre script en ne lançant qu'un unique gladiateur (`gladiateur.sh 1`).
2. Compléter votre script pour qu'il affiche "`X: Morituri te salutant`" lorsqu'il reçoit un signal `USR1`. Tester votre script en lançant un gladiateur dans un terminal (`./gladiateur.sh 1`), et en envoyant un signal `USR1` au gladiateur avec la commande `kill` **dans un autre terminal**.
3. Compléter votre script pour qu'il termine le processus en renvoyant un code de retour vrai (0) juste après avoir affiché "`X: Morituri te salutant`".
4. Vous devez remarquer que lorsque vous envoyez un signal `USR1`, votre processus ne reçoit le signal qu'après avoir terminé son attente de 5 secondes. Ce comportement est dû au fait que la commande interne `sleep` masque les signaux pendant qu'elle s'exécute. De façon à éviter cette attente, nous utilisons le fait que la commande `wait`, elle, peut être interrompue par un signal. Au lieu de lancer la commande `sleep` en avant plan, lancez-la en arrière plan, et utilisez `wait $!` pour attendre la fin de la commande `sleep` (le `$!` permet de n'attendre que la fin de la dernière commande, c.-à-d., `sleep`, et non celle de tous les enfants). Testez votre programme avec un unique gladiateur et vérifiez que le processus n'attend plus la fin de la commande `sleep`.

5.2 - Le problème de la terminaison

Le but ici est de vous montrer qu'il est difficile de terminer plusieurs gladiateurs.

1. Lancez deux gladiateurs avec la commande `./gladiateur.sh 2`. Envoyez un signal `USR1` a un des gladiateurs. Que constatez vous ?
2. Débarrassez-vous du second gladiateur.
3. Lancez deux gladiateurs avec la commande `./gladiateur.sh 2`. Saisissez la combinaison de touches `CTRL+c`. Que constatez-vous ?
4. Le cas échéant, débarrassez-vous des gladiateurs qui seraient encore en train de s'exécuter.

5.3 - Retrouver les PIDs des gladiateurs

Comme vous l'avez vu, il est difficile de terminer plusieurs gladiateurs car lorsque vous envoyez un signal, il n'est envoyé qu'à un seul gladiateur. Vous allez résoudre le problème avec le processus César. César, qui s'ennuie souvent au Colisée, apprécie d'envoyer des `USR1` à tous les gladiateurs pour les tuer. Pour connaître les PIDs des gladiateurs, César lit un parchemin nommé `arene.txt`, dans lequel se trouve un PID de gladiateur par ligne.

1. Un gladiateur doit enregistrer son PID dans le fichier `arene.txt` de façon à ce que César puisse le trouver. Modifier le script `./gladiateur.sh` pour qu'il ajoute son PID au fichier `arene.txt` après avoir vérifié que les paramètres sont corrects. Tester votre script en lançant deux fois un seul gladiateur que vous interromprez avec un signal `INT`, et en vérifiant que `arene.txt` contient bien les PID des gladiateurs :

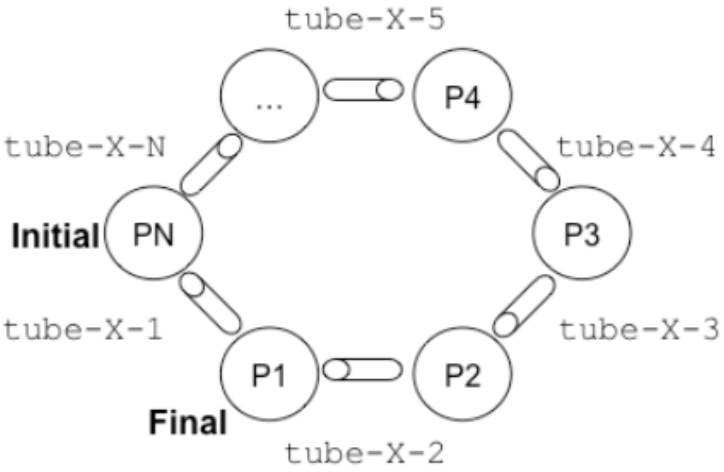
```
> ./gladiateur.sh 1
Processus 2460 démarre avec le processus initial 2460
  Fin de chaîne
2460: Ave César
2460: Ave César
^C
> cat arene.txt
2460
> ./gladiateur.sh 1
Processus 2473 démarre avec le processus initial 2473
  Fin de chaîne
2473: Ave César
^C
> cat arene.txt
2460
2473
```

2. Avant d'envoyer des signaux, écrire un script `cesar.sh` qui :
   — affiche un message d'erreur et renvoie faux si `arene.txt` n'existe pas,
   — lit ligne à ligne `arene.txt`, et affiche chaque ligne sur la sortie standard,
   — supprime le fichier `arene.txt` après avoir lu chaque ligne,
   — renvoie une code retour vrai.
   Tester votre script avec le fichier `arene.txt` que vous avez généré à la question précédente.
3. Au lieu d'afficher les PIDs des gladiateurs, `cesar.sh` doit maintenant envoyer un `USR1` à chaque gladiateur enregistré dans `arene.txt`. Modifier le script `cesar.sh` en conséquence.
4. Utiliser maintenant le script `cesar.sh` pour terminer proprement tous les gladiateurs lorsque l'utilisateur saisie `CTRL+c` ou utilise la commande `kill`. Modifier `gladiateur.sh` de façon à lancer `cesar.sh` à la réception des signaux `INT` et `TERM`.

Vous venez d'écrire votre premier protocole de terminaison.

6 - La boîte à Meuh (≃2h00)

Vous devez transformer "Le Colisée" en "Boîte à Meuh", c'est-à-dire que les processus vont afficher `Meuh` les uns après les autres. Même si afficher `Meuh` sur le terminal peut sembler quelque peu inutile, l'algorithme que vous allez mettre en œuvre est souvent utilisé dans les systèmes répartis et réseaux pour synchroniser un ensemble de processus. Par exemple, cet algorithme est utilisé pour construire le protocole *Token-Ring* qui permet un accès équitable à une infrastructure réseau.



Techniquement, les processus sont organisés en anneau, comme présenté dans la figure ci-dessus. Chaque processus de la chaîne crée un **tube nommé** `tube-X-N`, où `X` est le PID du processus initial (stocké dans la variable `pidInitial`) et `N` l'argument donné au script, c.-à-d., le nombre de processus restant à créer dans la chaîne. Pendant l'exécution, les processus communiquent soit avec leur prédécesseur en accédant au tube du prédécesseur, soit avec leur successeur, en accédant à leur propre tube.

À partir de cette topologie (c'est-à-dire de cette architecture), les processus échangent un "jeton", symbolisé par le mot `Meuh`. Un processus reçoit le jeton lorsqu'il reçoit le mot `Meuh` via le tube de son prédécesseur. Le processus conserve ensuite le jeton pendant une seconde avant de le passer à son successeur en écrivant `Meuh` dans son tube.

1. Pour commencer, il faut que chaque processus trouve *(i)* le nom du tube pour communiquer avec son prédécesseur et *(ii)* le nom du tube pour communiquer avec son successeur.
   — Le nom du tube successeur est toujours `tube-$pidInitial-N`, où `pidInitial` est le PID du processus initial et `N` l'argument du script (*i.e.*, `$1`).
   — Pour le nom du tube prédécesseur :
   — si le processus est le processus initial (c.-à-d., le seul processus démarrant avec une variable `pidInitial` non initialisée), ce nom est `tube-$$-1`, où `$$` est le PID du processus courant, puisque le processus courant est le processus initial,
   — sinon, le nom du tube est `tube-$pidInitial-M`, où M vaut `$1+1`,

   Après avoir copié `gladiateur.sh` en `meuh.sh`, modifier votre script pour :
   — stocker dans les variables `pred` et `succ` les noms des tubes du prédécesseur et du successeur,
   — modifier l'affichage Processus `$$` démarre avec le processus initial `$pidInitial` de façon à afficher les noms de ces tubes.

   Remarque : On ne vous demande pas de créer les tubes à cette question.
2. Modifier votre script de façon à ce que chaque processus de la chaîne crée son tube successeur après avoir identifié les noms des tubes successeurs et prédécesseurs. Pensez à vérifier que le tube n'existe pas déjà avec `[[ ! -p "$succ" ]]`.
3. Avant d'aller plus loin, il faut être capable de supprimer les tubes créés par notre application. Modifier `cesar.sh` de façon à supprimer chacun des tubes créés. Vous pouvez remarquer que, comme `meuh.sh` exporte `pidInitial`, cette variable est positionnée dans `cesar.sh`. Pour cette raison, il suffit de supprimer tous les fichiers dont le nom commence par `tube-$pidInitial-` dans `cesar.sh`. Pensez à utiliser l'option `-f` de la commande `rm`, qui évite de demander une confirmation à l'utilisateur. **ATTENTION à utiliser cette option avec précaution ! Soyez sûre à 100% de la commande avant de la lancer !**
4. Pour accéder aux tubes, nous utilisons des redirections avancées. En effet, des redirections simples ouvrent et ferment continuellement les tubes, ce qui entraîne des erreurs ou des blocages lorsque les tubes sont fermés pendant qu'il existe des interlocuteurs. Avant la boucle principale qui affiche "`Ave César`", ouvrez en lecture/écriture les tubes prédécesseurs et successeurs (2 ouvertures différentes). Vous devriez remarquer que vous avez un message d'erreur du type : "`mkfifo: tube-3665-1: File exists`". Vous vous occuperez de ce message à la question suivante.
5. Le message d'erreur que vous avez est dû à un problème de *synchronisation*. Le processus initial arrive à atteindre l'ouverture du tube du prédécesseur avant que le processus final ait eu le temps de créer son tube. Comme l'ouverture par le processus initial crée un fichier normal, la création du tube dans le processus final échoue avec un message d'erreur. Pour éviter ce problème, il faut donc ouvrir le tube du prédécesseur uniquement si le tube existe. Avant d'ouvrir le tube du prédécesseur, ajoutez donc une boucle qui attend tant que le tube n'existe pas et qui dort une seconde à chaque itération de la boucle. Vérifiez que le message d'erreur a bien disparu.
6. Pour mieux voir comment fonctionne le protocole, nous lançons chaque processus dans un terminal différent. Remplacez la création récursive d'un processus avec `meuh.sh K`, où `K` est égal au nombre de processus restant à créer dans la chaîne, par `xterm -reverse -e $0 K &` (l'option `reverse` crée un terminal blanc

sur fond noir et est optionnelle, alors que l'option `-e` indique à `xterm` le nom du programme à exécuter dans le terminal). Vous pouvez admirer votre protocole de terminaison qui permet de fermer tous les terminaux en saisissant CTRL+C dans n'importe quel terminal.

7. Nous mettons maintenant en place notre boîte à Meuh. Dans la boucle principale, remplacez l'affichage de "`Ave César`" et l'attente de 5 secondes par :
   — une lecture d'une ligne à partir du tube prédécesseur,
   — l'affichage de la ligne lue,
   — une attente d'une seconde,
   — l'écriture de la ligne lue dans le tube successeur.
   Pour amorcer votre boîte à Meuh, écrivez `Meuh` dans un des tubes à partir d'un autre terminal.
8. Amorcer la boîte à Meuh à partir d'un autre terminal est relativement fastidieux. Pour cette raison, c'est le processus final qui va générer le premier jeton. Modifier votre script pour que le processus final (celui qui a pour paramètre 1) écrive `Meuh` dans le tube de son successeur juste avant d'exécuter la boucle principale du programme.

Félicitations, vous venez d'écrire votre premier protocole multiprocessus complexe !